
AVR[®] Low-Power Techniques

Introduction

Author: Per Andreas Gulbrandsen, Microchip Technology Inc.

This application note will discuss several techniques available to help limit the power consumption of AVR[®] microcontrollers. It covers techniques and concepts such as sleep modes, the choice of oscillator and operating frequency, using the Event System (EVSYS), Core Independent Peripherals (CIPs) and the Brown-out Detector (BOD), and what to do with unused pins.

Table of Contents

Introduction.....	1
1. Relevant Devices.....	3
2. Conceptual Application.....	6
3. Active Peripherals.....	7
4. Operating Modes.....	8
4.1. Overview.....	8
4.2. Active Mode Operation.....	8
4.3. Sleep modes.....	9
4.4. Applying Operating Modes to the Conceptual Application.....	9
4.5. Execution of the Sleep Instruction and Shared Variables Between ISR and Main.....	9
5. Event System.....	11
5.1. Overview.....	11
5.2. Applying Events to Conceptual Application.....	11
6. Core Independent Peripherals.....	12
6.1. Applying a Core Independent Peripheral to Conceptual Application.....	12
7. Choice of Oscillator.....	13
7.1. Applying Choice of Oscillator to Conceptual Application.....	13
8. BOD.....	14
8.1. BOD Sleep Mode Operation.....	14
8.2. Applying BOD to Conceptual Application.....	14
9. Unused Pins.....	15
10. Voltage Regulator Configuration.....	16
10.1. Applying Voltage Regulator Configuration to Conceptual Application.....	16
11. Revision History.....	17
The Microchip Website.....	18
Product Change Notification Service.....	18
Customer Support.....	18
Microchip Devices Code Protection Feature.....	18
Legal Notice.....	18
Trademarks.....	19
Quality Management System.....	20
Worldwide Sales and Service.....	21

1. Relevant Devices

This section lists the relevant devices for this document. The following figures show the different family devices, laying out pin count variants and memory sizes:

- Vertical migration upwards is possible without code modification, as these devices are pin-compatible and provide the same or more features. Downward migration on tinyAVR® 1-series devices may require code modification due to fewer available instances of some peripherals.
- Horizontal migration to the left reduces the pin count and, therefore, the available features
- Devices with different Flash memory sizes typically also have different SRAM and EEPROM

Figure 1-1. tinyAVR® 0-series Overview

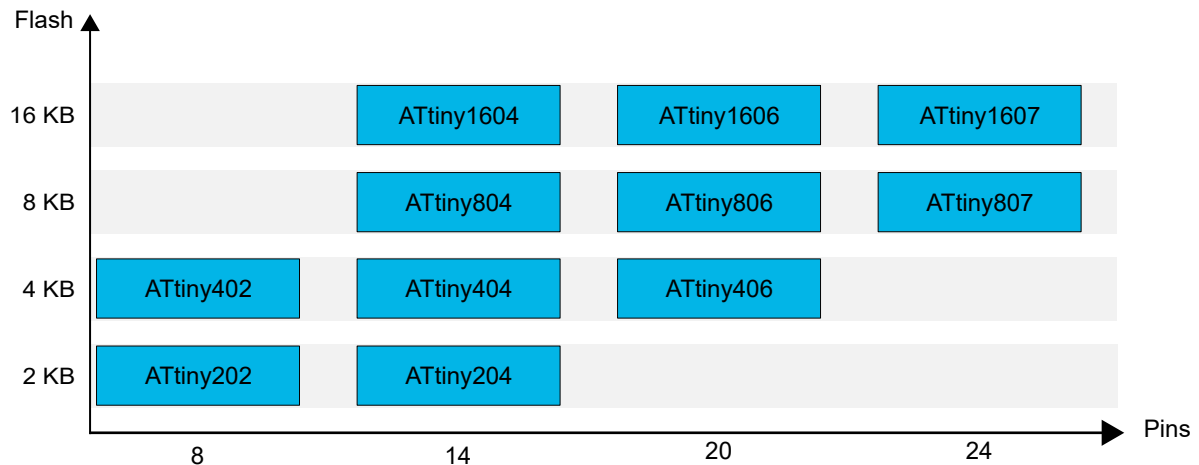


Figure 1-2. tinyAVR® 1-series Overview

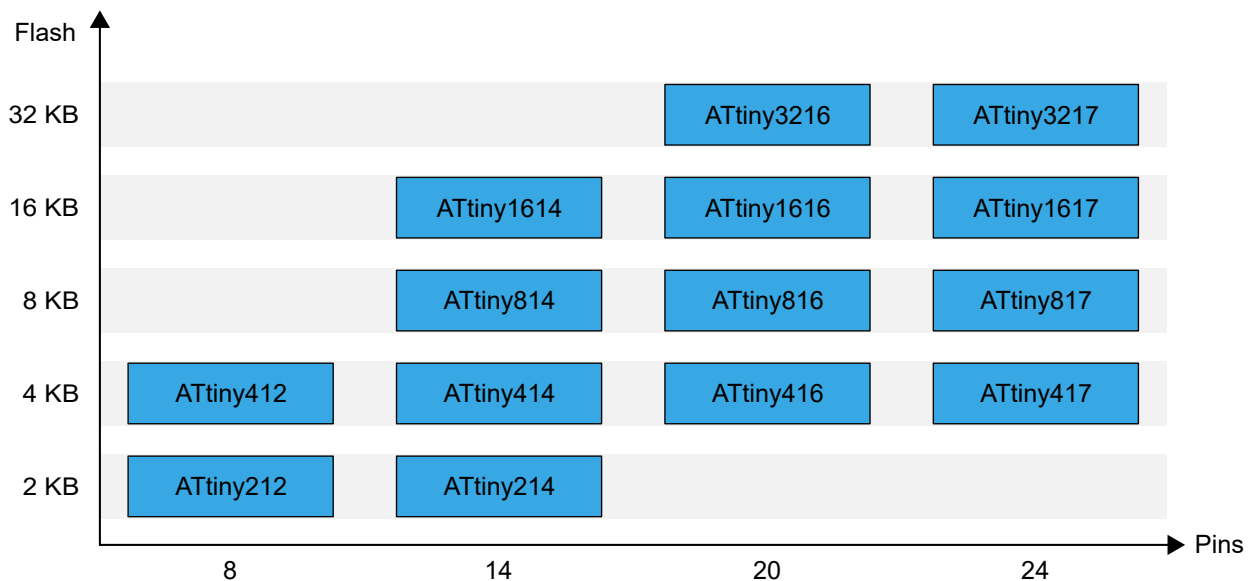


Figure 1-3. tinyAVR® 2 Family Overview

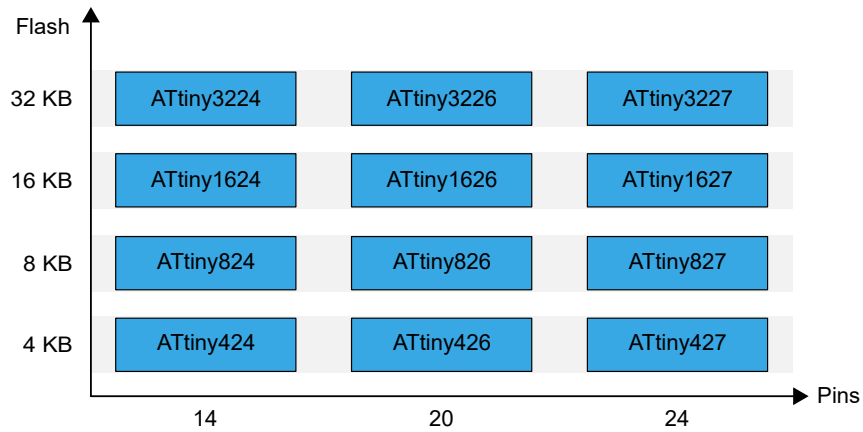


Figure 1-4. megaAVR® 0-series Overview

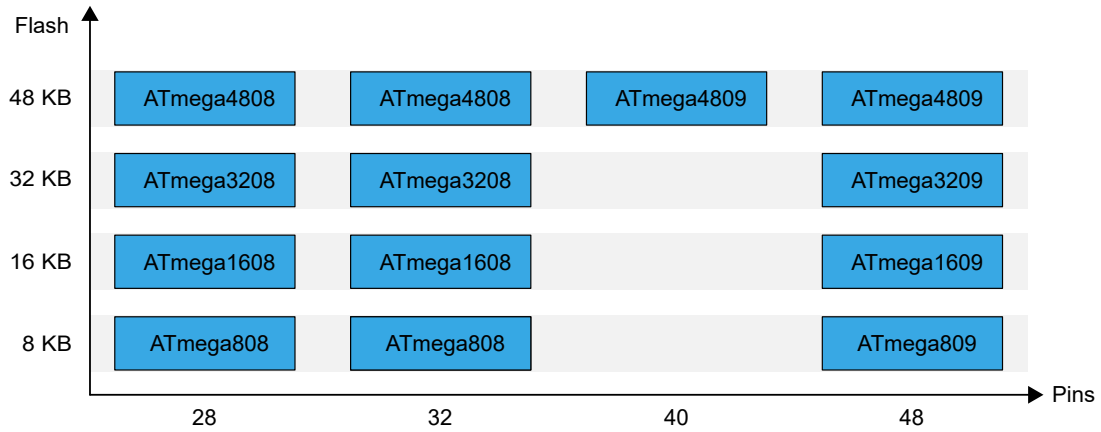


Figure 1-5. AVR® DA Family Overview

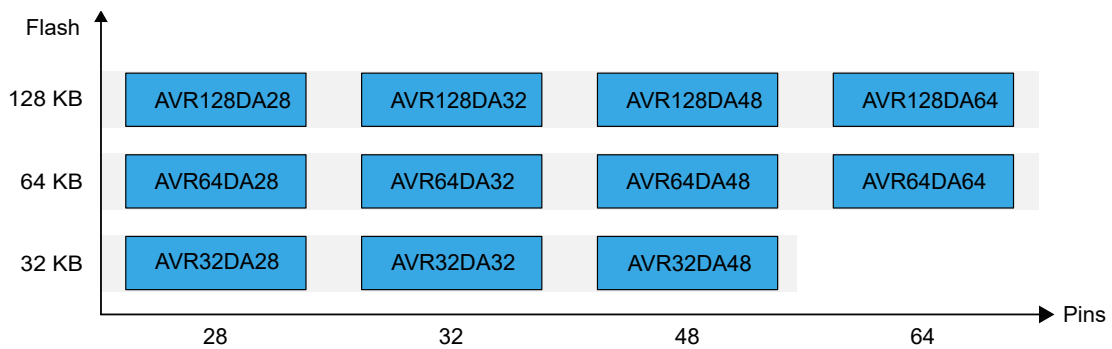
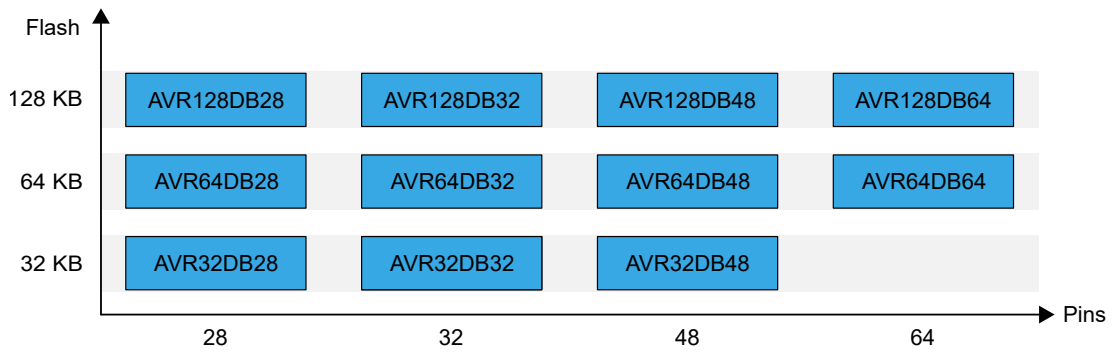


Figure 1-6. AVR® DB Family Overview



2. Conceptual Application

This application note will discuss several techniques available to help limit the power consumption. To illustrate this, a conceptual application is introduced, and the techniques will be applied to this application.

The conceptual application is a very simple application that monitors an external temperature sensor connected to a pin. In its basic form, the application busy-spins for one second then measures the value of the sensor using the Analog-to-Digital Converter (ADC) and compares this value to two predefined threshold values. Two threshold values are used to create a window, in order to avoid constant switching. Based on this comparison, several actions can be taken, such as turning the heater OFF if the temperature is above the upper threshold value or turning the heater ON if the temperature is below the lower threshold value. As long as the temperature is inside the window, no adjustments are made. For simplicity, in this application, these actions are turning a pin ON or OFF.

3. Active Peripherals

When peripherals are enabled, they can impact power consumption even if they are not active. Therefore, a developer should make sure that only peripherals in use are enabled. On the [Relevant Devices](#) this is intuitive, as all peripherals are disabled by default.

On earlier AVR® devices, the Power Reduction Register (PRR) register has commonly been used to enable or disable peripherals. On the [Relevant Devices](#), this register does not exist. Instead, in the register interface of each peripheral, there is an Enable bit. This, plus a more advanced clock system, with better clock gating, meaning that fewer gates are clocked in an unused clock path, replaces the PRR functionality. It is also more intuitive and user-friendly, as the Enable functionality is placed within the peripheral in question.

The peripheral operation can also be affected by sleep modes. This is discussed in [4. Operating Modes](#).

4. Operating Modes

Most applications do not require the CPU to run continuously after start-up. For such an application, use sleep modes to lower power consumption.

4.1 Overview

Sleep modes are used to shut down peripherals and clock domains in the device in order to save power. The Sleep Controller (SLPCTRL) controls and handles the transitions between Active and Sleep mode.

There are in total four modes available:

- One Active mode in which the software is executed
- Three Sleep modes:
 - Idle
 - Standby
 - Power-Down

All Sleep modes are available and can be entered from Active mode. In active mode, the CPU is executing application code. When the device enters Sleep mode, program execution is stopped and interrupts or a Reset is used to wake the device again. The application code decides which Sleep mode to enter and when.

Interrupts are used to wake the device from sleep. The available interrupt wake-up sources depend on the configured Sleep mode. When an interrupt occurs, the device will wake up and execute the interrupt service routine before continuing normal program execution from the first instruction after the `SLEEP` instruction. Any Reset will take the device out of a Sleep mode.

The content of the register file, SRAM and registers are kept during Sleep. If a Reset occurs during Sleep, the device will Reset, start, and execute from the Reset vector.

4.2 Active Mode Operation

In Active mode all clocks are active, and the CPU is running. Power consumption in active mode is proportional to the operating frequency of the system clock. Applications that do not demand high-frequency operation can benefit from lowering the system clock.

Even though the rule of thumb is that lower frequency equals lower power, in some cases the opposite is true. Running at a higher clock frequency could mean that the CPU will finish its task faster, and return back to sleep quicker, resulting in a lower total power consumption. Imagine an application that wakes up from Standby sleep mode periodically and performs some calculations. If these calculations are done at a higher clock frequency the system will spend less time in Active and more time in Standby sleep mode, thus lowering the total power consumption. An example of the opposite could be where an operation such as USART communication is preventing the CPU from sleeping. In this situation, the timing of the USART transaction is the limiting factor, and higher CPU frequency will not help reduce time spent in Active mode.

Running the system at one static frequency can be sufficient for many applications. For some applications, there are advantages to using dynamic frequency scaling. As the operating frequency of the device can be changed on the fly, some parts of an application can execute at a lower frequency, while other parts can be executed at a higher frequency. For example, this can be useful in applications that are very computationally heavy for short periods. An example of this can be an application that mainly sleeps while collecting data, and periodically performs heavy computations on the sampled data. Using a higher clock frequency to perform the computations can reduce the total power consumption, even though the consumption when the computations are being performed is higher, because of the shorter duration of the computation period.

Note that this technique has a trade-off. As there is a cost related to executing the instructions needed to change the frequency, the code executed at higher frequency must compensate for this. The savings from executing code faster must outweigh the cost of adding code to change the frequency. Also, note that the frequency must most likely be changed both when entering and exiting the part of the application selected for high-frequency execution.

A convenient way of doing dynamic frequency scaling is to use the Main Clock Prescaler. This prescaler allows the clock to be scaled before going to the CPU and peripherals. Note that as this prescaler affects the clocks for both CPU and peripherals, any peripherals configured for a certain clock speed could malfunction. Examples of this could be violating the maximum clock speed of the ADC or changing the BAUD rate of the USART when changing the peripheral clock. Refer to the Clock Controller (CLKCTRL) section in the corresponding data sheet for details.

Creating a power budget can be a useful exercise when developing a low-power application. First, calculate the power consumption for each operating mode (use the Electrical Characteristics section in the data sheet). Then, calculate how much time is spent in each mode. Lastly, using the numbers from the two previous steps, calculate the average and peak consumption, allowing the developer to calculate, for example, the battery life for that application. It can also be a valuable tool in profiling and optimizing the application.

4.3 Sleep modes

In addition to Active mode, there are three different sleep modes, with decreasing power consumption and functionality. In Idle mode, the CPU stops executing code, but no peripherals are disabled. Also, all interrupt sources can wake the device. In Standby mode, on the other hand, the user can configure peripherals to be enabled or not; the power consumption is thus highly dependent on what functionality is enabled. In Power-Down mode, only a few peripherals are active, and there are also fewer wake-up sources. Refer to the Sleep Controller (SLPCTRL) section in the corresponding data sheet for the complete description of the sleep modes, which peripherals are active, and available wake-up sources.

4.4 Applying Operating Modes to the Conceptual Application

As the [2. Conceptual Application](#) is inactive while waiting for another sample to be converted, great amounts of power can be saved by sleeping while waiting. In order to do this, the application needs a wake-up source. This is easily done by setting up the RTC to interrupt and wake the system once every second. When a wake-up happens, the CPU can initiate an ADC conversion, compare the result, take appropriate action if any, and turn back to sleep.

If we roughly estimate that the system will be awake for 1 ms per second for each sample (starting the system clock and CPU, starting the ADC conversion, waiting for ADC interrupt, reading out the result and comparing it, possibly toggling a pin, and going back to sleep), the expected time in Active mode is 0.1%. This is compared to not using sleep modes, and having the CPU stay continuously awake. The expected power consumption will be slightly above 0.1%, as power consumption in Sleep mode must also be taken into account.

Dynamic clock scaling can also be applied to the conceptual application. While the CPU is waiting for the ADC to complete there is no need for high CPU frequency. Based on the output value, the application decides if it needs to take action. If this is the case, the system can switch to a higher clock frequency to be able to complete these actions faster, and thus reduce the time spent in Active mode. However, as only a few instructions are executed after the comparison operation, there is no benefit to using this technique in the conceptual application.

4.5 Execution of the Sleep Instruction and Shared Variables Between ISR and Main

The potential problem with the sleep instruction is that an ISR can have executed right before the execution of the sleep instruction. If that ISR wrote to a flag which then needs to be handled in the main loop, it will not be handled until the next interrupt occurs and causes the device to wake up. The following code shows an example where this can happen:

```
volatile uint8_t shared_flag;
ISR(perip_vector){
    uint8_t i_flags = PERIP_INTFLAGS;
    shared_flag = 0x01;
    PERIP_INTFLAGS = i_flags;
}

main(){
    while(1){
        cli();
        if(shared_flag == 0x01){
            shared_flag &= ~0x01;
            sei();
        }
    }
}
```

```
        handle_event_etc();
    }
    sei();
    sleep();
}
}
```

In this case, the *shared_flag* is protected from being altered by the ISR while it is being checked and cleared. However, if the ISR writes to the flag after the first *sei()*, inside the if statement, the sleep instruction will be executed without the *shared_flag* being checked again. As such, the events that need to happen in *main()* will not happen until the next time an interrupt occurs, waking the device from sleep.

To handle all flags inside the main loop, use the following code:

```
main() {
    while(1) {
        cli();
        if(shared_flag == 0) {
            sei();
            sleep();
        }
        sei();
        //handle shared_flags and run application
    }
}
```

This code ensures that no shared flags between ISRs and the main loop go unhandled before going to sleep. It does so because any instruction executing after the *sei()* instruction will be allowed to execute before a jump into a pending interrupt. The *shared_flag* variable can be shared between multiple ISRs. Note that any read-modify-write of the *shared_flag* inside the main loop will have to be protected by *cli()* and *sei()* as shown in the code below:

```
cli();
shared_flag &= ~0x01;
sei();
```

This makes the read-modify-write an atomic action, that is, it cannot be interrupted by any ISRs. However, avoid this by using General Purpose Registers (GPR) as shared variables between ISRs. These registers allow the compiler to compile any single-bit modification to one line of assembly code that will execute in a single CPU cycle. In comparison, using a read-modify-write of a *uint8_t* variable will take three cycles, or five cycles, when taking the *cli()* and *sei()* instructions into account.

5. Event System

5.1 Overview

The Event System (EVSYS) enables direct peripheral-to-peripheral signaling. It allows a change in one peripheral (the event generator) to trigger actions in other peripherals (the event users) through event channels, without using the CPU. It is designed to provide a short and predictable response time between peripherals, allowing for autonomous peripheral control and interaction, and for synchronized timing of actions in several peripheral modules. Thus, the EVSYS peripheral makes it possible to implement Core Independent Peripherals (CIPs). Also, it is a powerful tool for reducing the complexity, size, and execution time of the software.

A change of the event generator's state is referred to as an event and usually corresponds to one of the peripheral's interrupt conditions. Events can be forwarded directly to other peripherals using the dedicated event routing network. The routing of each channel is configured in software, including event generation and use.

Only one event signal can be routed on each channel. Multiple peripherals can use events from the same channel.

The EVSYS can connect peripherals such as ADCs, analog comparators, I/O PORT pins, the real-time counter, timer/counters, and the configurable custom logic peripheral. Events can also be generated from software.

5.2 Applying Events to Conceptual Application

The [2. Conceptual Application](#) can benefit from using Events. The RTC, which times the operation, can use an event to signal to the ADC that a conversion must be done, instead of waking the CPU to start a conversion. This allows the time spent in sleep mode to be slightly extended, as the CPU can now sleep while the ADC is converting. As the ADC conversion is very quick (13 ADC cycles, 52 CPU cycles), and the reduction of time spent in Active mode is equally short, the reduction in power consumption will be small. The gain in using events will be more visible when adding ADC window mode (explained in [6. Core Independent Peripherals](#)).

6. Core Independent Peripherals

Core Independent Peripherals (CIPs) are peripherals designed to handle their tasks without any intervention from the CPU, freeing the CPU to do other tasks in parallel or, for our purposes, enter a low-power mode. Other advantages of CIPs include providing short and predictable response times between peripherals and reducing the complexity and execution time of the software. In [5. Event System](#) we have already seen an example of a CIP. Other examples include Configurable Custom Logic (CCL), Timer/Counter A and B (TCA/TCB), Real Time Counter (RTC), Analog-to-Digital Converter (ADC) and CRCSCAN.

Some peripherals can process incoming data without waking the CPU. Depending on the incoming data, the peripherals can decide if the CPU should wake up, or if there is no further action to take. Two examples of such peripherals are the Two-Wire Interface (TWI) and the ADC. The TWI supports Wake on Address Match, where it examines the start of an incoming data frame, and only wakes the CPU if the address of the incoming frame matches its own address. The ADC supports Window mode, where each sample is compared with two values, representing a window. Based on configuration, the ADC can wake the CPU if the sample is below, above, inside, or outside the window.

6.1 Applying a Core Independent Peripheral to Conceptual Application

For example, implementing ADC window mode in the conceptual application could potentially allow the CPU to extend the time in Standby sleep mode. The ADC itself can compare the converted value, and check if it is below the window (if the heater is OFF) or above the window (if the heater is ON). The CPU will only wake up when it needs to turn the heater ON or OFF. In a perfectly stable environment, the CPU would never wake up, causing the total power consumption to be equal to the sleep mode consumption. In a constantly fluctuating environment, where the temperature alternates between below and above the thresholds for every sample (i.e., every second), the power consumption will be the same as when the CPU wakes up to do the comparison for every converted ADC sample.

7. Choice of Oscillator

As already mentioned, the system clock greatly impacts the power consumption of the system. In addition, the choice of oscillator also impacts the power consumption. As a general guideline, this is a tradeoff between accuracy and consumption. E.g., when selecting a 32 kHz clock, the internal 32K Ultra-Low Power (ULP) oscillator will offer drastically lower power consumption than an external crystal, but the accuracy is also worse. If the application is not dependent on high accuracy, choosing the ULP oscillator will help lower the power budget.

Allowing the clock sources to be disabled during sleep modes will lower the power consumption of the oscillator. However, for oscillators with a significant wake-up time, this will shorten the amount of time spent in sleep mode. For applications requiring high-accuracy timing, enabling oscillators to run in standby by writing the corresponding RUNSTDBY bit (e.g., XOSC32KCTRLA.RUNSTDBY for the external 32K oscillator) will eliminate the start-up time and thus improve the timing. Refer to the Sleep Controller (SLPCTRL) and Clock Controller (CLKCTRL) sections in the corresponding data sheet for details on the wake-up time.

There are techniques that can be used to improve the accuracy of the 32K ULP. By doing periodic run-time calibration, using a high-accuracy external crystal, the accuracy of the internal 32K ULP oscillator can be greatly improved. Refer to the application note on precise, ultra-low power timing using periodic enabling of the 32.768 kHz external crystal oscillator for recalibration of the ULP internal oscillator.

7.1 Applying Choice of Oscillator to Conceptual Application

The [2. Conceptual Application](#) does not require great accuracy, and can, therefore, use the internal 32K ULP oscillator as the clock source of the RTC to keep track of time while sleeping.

8. BOD

The Brown-out Detector (BOD) protects the microcontroller when the supply voltage falls below its operating threshold by resetting the device. This keeps the microcontroller in a defined state when the V_{CC} is below its operating threshold. The BOD is not important to the microcontroller while it is in sleep mode but it is extremely important when it wakes up. Therefore, as a rule, most microcontrollers keep the BOD active during sleep mode and it contributes substantially to sleep mode power consumption.

The solution to this problem is to have the microcontroller shut down the BOD when it enters sleep mode and starts it again just before leaving sleep mode. This approach ensures the BOD is functioning when it is needed without any power consumption penalty while in sleep mode. However, this approach does add to the start-up time of the device, as the BOD needs additional start-up time.

8.1 BOD Sleep Mode Operation

The BOD can be configured in four ways for Active operation: Disabled, Enabled, Sampled and Enabled with wake-up halted until BOD is ready.

- Disabled: The BOD is disabled.
- Enabled: The BOD is continually sampling the supply voltage and comparing it to the selected threshold.
- Sampled: The supply voltage is sampled and compared to the threshold at either 125 Hz or 1 kHz.
- Enabled with wake-up halted until BOD is ready: The BOD will sample the supply voltage when a wake-up occurs, and wake-up will be halted until the BOD signals that the supply voltage is above the selected threshold. This adds to the wake-up time. In active mode, the BOD is continually sampling the supply voltage and comparing it to the selected threshold.

For Sleep operation, there are three configurations: Disabled, Enabled, and Sampled.

The Active operation can only be configured using fuses, while the Sleep operation can be configured using both fuses and registers. Refer to the corresponding data sheet for details.

8.2 Applying BOD to Conceptual Application

The [conceptual application](#) does not need a BOD in sleep. Disabling the BOD in sleep will reduce the power consumption.

In active mode, the conceptual application can use the Enabled with wake-up halted mode. This configuration means that the BOD will monitor the supply current when needed (active mode), but will not be active in sleep mode. Supply current is checked when a wake-up occurs, so that normal operation is safe. There will be a minor penalty of added wake-up time. But, as the conceptual application is not time-critical, this is tolerated.

9. Unused Pins

All digital I/O pins are by default floating to avoid hardware conflicts. However, since the pins have digital input buffers it is important to ensure that the level on an I/O pin is well-defined to avoid sporadic internal switching and leakage. The leakage caused by floating I/O is relatively small and is mainly observable in sleep, but can be minimized by enabling the internal pull-up. Using an external pull-up is also an option.

In addition, disabling the digital input buffer on unused pins will further lower the power consumption, which is also true for pins connected to an analog peripheral, e.g., the ADC. Disabling of the digital input buffer and enabling the internal pull-up can be done in the PINCTRLn registers for the individual ports. Refer to the corresponding data sheet for details.

10. Voltage Regulator Configuration

For AVR DA and DB devices, there is another technique to be aware of for applications requiring high-accuracy timing. The voltage regulator can be configured in the Sleep Controller (SLPCTRL) to balance the power consumption with the wake-up time and with the maximum clock speed. In its default configuration, the regulator is in Normal (AUTO) mode. In this mode, the regulator will consume less power when sleeping in Standby/Power-Down mode or when the 32 kHz is the only oscillator enabled, but the wake-up time will be longer. Instead, if the regulator is configured to Performance (FULL) mode, it will use maximum power consumption in all modes. But the wake-up time will be faster from sleep modes, which is a tradeoff between wake-up time and power consumption, and, as discussed in [7. Choice of Oscillator](#), a longer wake-up time will shorten the amount of time spent in sleep mode. Configuring the Voltage Regulator to Performance mode will shorten the wake-up time while still having the clock source disabled during sleep mode. For exact values for power consumption and start-up times, refer to the Electrical Characteristics section in the corresponding data sheet.

10.1 Applying Voltage Regulator Configuration to Conceptual Application

The [2. Conceptual Application](#) does not require great accuracy. It can, therefore, use the Voltage Regulator in Normal (AUTO) mode. Also, note that configuring the Voltage Regulator is relevant only when using a DA or DB device.

11. Revision History

Doc. Rev.	Date	Comments
D	01/2022	• Updated the section on relevant devices • Changed the section on Sleepwalking to Core Independent Peripherals and updated it • Added a new section on Voltage Regulator Configuration for DA and DB devices • Updated the sections on Sleep modes and Event system
C	10/2018	Updated list of tinyAVR and megaAVR relevant devices
B	01/2018	New sections: • <i>tinyAVR 0-series</i> • <i>megaAVR 0-series</i> • <i>Execution of the Sleep Instruction and Shared Variables Between ISR and Main</i>
A	07/2017	Initial document release.

The Microchip Website

Microchip provides online support via our website at www.microchip.com/. This website is used to make files and information easily available to customers. Some of the content available includes:

- **Product Support** – Data sheets and errata, application notes and sample programs, design resources, user's guides and hardware support documents, latest software releases and archived software
- **General Technical Support** – Frequently Asked Questions (FAQs), technical support requests, online discussion groups, Microchip design partner program member listing
- **Business of Microchip** – Product selector and ordering guides, latest Microchip press releases, listing of seminars and events, listings of Microchip sales offices, distributors and factory representatives

Product Change Notification Service

Microchip's product change notification service helps keep customers current on Microchip products. Subscribers will receive email notification whenever there are changes, updates, revisions or errata related to a specified product family or development tool of interest.

To register, go to www.microchip.com/pcn and follow the registration instructions.

Customer Support

Users of Microchip products can receive assistance through several channels:

- Distributor or Representative
- Local Sales Office
- Embedded Solutions Engineer (ESE)
- Technical Support

Customers should contact their distributor, representative or ESE for support. Local sales offices are also available to help customers. A listing of sales offices and locations is included in this document.

Technical support is available through the website at: www.microchip.com/support

Microchip Devices Code Protection Feature

Note the following details of the code protection feature on Microchip products:

- Microchip products meet the specifications contained in their particular Microchip Data Sheet.
- Microchip believes that its family of products is secure when used in the intended manner, within operating specifications, and under normal conditions.
- Microchip values and aggressively protects its intellectual property rights. Attempts to breach the code protection features of Microchip product is strictly prohibited and may violate the Digital Millennium Copyright Act.
- Neither Microchip nor any other semiconductor manufacturer can guarantee the security of its code. Code protection does not mean that we are guaranteeing the product is "unbreakable". Code protection is constantly evolving. Microchip is committed to continuously improving the code protection features of our products.

Legal Notice

This publication and the information herein may be used only with Microchip products, including to design, test, and integrate Microchip products with your application. Use of this information in any other manner violates these terms. Information regarding device applications is provided only for your convenience and may be superseded by updates. It is your responsibility to ensure that your application meets with your specifications. Contact your local Microchip sales office for additional support or, obtain additional support at www.microchip.com/en-us/support/design-help/client-support-services.

THIS INFORMATION IS PROVIDED BY MICROCHIP "AS IS". MICROCHIP MAKES NO REPRESENTATIONS OR WARRANTIES OF ANY KIND WHETHER EXPRESS OR IMPLIED, WRITTEN OR ORAL, STATUTORY OR OTHERWISE, RELATED TO THE INFORMATION INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A PARTICULAR PURPOSE, OR WARRANTIES RELATED TO ITS CONDITION, QUALITY, OR PERFORMANCE.

IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE, INCIDENTAL, OR CONSEQUENTIAL LOSS, DAMAGE, COST, OR EXPENSE OF ANY KIND WHATSOEVER RELATED TO THE INFORMATION OR ITS USE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL CLAIMS IN ANY WAY RELATED TO THE INFORMATION OR ITS USE WILL NOT EXCEED THE AMOUNT OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THE INFORMATION.

Use of Microchip devices in life support and/or safety applications is entirely at the buyer's risk, and the buyer agrees to defend, indemnify and hold harmless Microchip from any and all damages, claims, suits, or expenses resulting from such use. No licenses are conveyed, implicitly or otherwise, under any Microchip intellectual property rights unless otherwise stated.

Trademarks

The Microchip name and logo, the Microchip logo, Adaptec, AnyRate, AVR, AVR logo, AVR Freaks, BesTime, BitCloud, CryptoMemory, CryptoRF, dsPIC, flexPWR, HELDO, IGLOO, JukeBlox, KeeLoq, KleeR, LANCheck, LinkMD, maXStylus, maXTouch, MediaLB, megaAVR, Microsemi, Microsemi logo, MOST, MOST logo, MPLAB, OptoLyzer, PIC, picoPower, PICSTART, PIC32 logo, PolarFire, Prochip Designer, QTouch, SAM-BA, SenGenuity, SpyNIC, SST, SST Logo, SuperFlash, Symmetricom, SyncServer, Tachyon, TimeSource, tinyAVR, UNI/O, Vectron, and XMEGA are registered trademarks of Microchip Technology Incorporated in the U.S.A. and other countries.

AgileSwitch, APT, ClockWorks, The Embedded Control Solutions Company, EtherSynch, Flashtec, Hyper Speed Control, HyperLight Load, IntelliMOS, Libero, motorBench, mTouch, Powermite 3, Precision Edge, ProASIC, ProASIC Plus, ProASIC Plus logo, Quiet- Wire, SmartFusion, SyncWorld, Temux, TimeCesium, TimeHub, TimePictra, TimeProvider, TrueTime, WinPath, and ZL are registered trademarks of Microchip Technology Incorporated in the U.S.A.

Adjacent Key Suppression, AKS, Analog-for-the-Digital Age, Any Capacitor, AnyIn, AnyOut, Augmented Switching, BlueSky, BodyCom, CodeGuard, CryptoAuthentication, CryptoAutomotive, CryptoCompanion, CryptoController, dsPICDEM, dsPICDEM.net, Dynamic Average Matching, DAM, ECAN, Espresso T1S, EtherGREEN, GridTime, IdealBridge, In-Circuit Serial Programming, ICSP, INICnet, Intelligent Paralleling, Inter-Chip Connectivity, JitterBlocker, Knob-on-Display, maxCrypto, maxView, memBrain, Mindi, MiWi, MPASM, MPF, MPLAB Certified logo, MPLIB, MPLINK, MultiTRAK, NetDetach, NVM Express, NVMe, Omniscient Code Generation, PICDEM, PICDEM.net, PICkit, PICtail, PowerSmart, PureSilicon, QMatrix, REAL ICE, Ripple Blocker, RTAX, RTG4, SAM-ICE, Serial Quad I/O, simpleMAP, SimpliPHY, SmartBuffer, SmartHLS, SMART-I.S., storClad, SQL, SuperSwitcher, SuperSwitcher II, Switchtec, SynchroPHY, Total Endurance, TSHARC, USBCheck, VariSense, VectorBlox, VeriPHY, ViewSpan, WiperLock, XpressConnect, and ZENA are trademarks of Microchip Technology Incorporated in the U.S.A. and other countries.

SQTP is a service mark of Microchip Technology Incorporated in the U.S.A.

The Adaptec logo, Frequency on Demand, Silicon Storage Technology, Symmcom, and Trusted Time are registered trademarks of Microchip Technology Inc. in other countries.

GestIC is a registered trademark of Microchip Technology Germany II GmbH & Co. KG, a subsidiary of Microchip Technology Inc., in other countries.

All other trademarks mentioned herein are property of their respective companies.

© 2022, Microchip Technology Incorporated and its subsidiaries. All Rights Reserved.

ISBN: 978-1-5224-9286-3

Quality Management System

For information regarding Microchip's Quality Management Systems, please visit www.microchip.com/quality.

Worldwide Sales and Service

AMERICAS	ASIA/PACIFIC	ASIA/PACIFIC	EUROPE
Corporate Office 2355 West Chandler Blvd. Chandler, AZ 85224-6199 Tel: 480-792-7200 Fax: 480-792-7277 Technical Support: www.microchip.com/support Web Address: www.microchip.com	Australia - Sydney Tel: 61-2-9868-6733 China - Beijing Tel: 86-10-8569-7000 China - Chengdu Tel: 86-28-8665-5511 China - Chongqing Tel: 86-23-8980-9588 China - Dongguan Tel: 86-769-8702-9880 China - Guangzhou Tel: 86-20-8755-8029 China - Hangzhou Tel: 86-571-8792-8115 China - Hong Kong SAR Tel: 852-2943-5100 China - Nanjing Tel: 86-25-8473-2460 China - Qingdao Tel: 86-532-8502-7355 China - Shanghai Tel: 86-21-3326-8000 China - Shenyang Tel: 86-24-2334-2829 China - Shenzhen Tel: 86-755-8864-2200 China - Suzhou Tel: 86-186-6233-1526 China - Wuhan Tel: 86-27-5980-5300 China - Xian Tel: 86-29-8833-7252 China - Xiamen Tel: 86-592-2388138 China - Zhuhai Tel: 86-756-3210040	India - Bangalore Tel: 91-80-3090-4444 India - New Delhi Tel: 91-11-4160-8631 India - Pune Tel: 91-20-4121-0141 Japan - Osaka Tel: 81-6-6152-7160 Japan - Tokyo Tel: 81-3-6880-3770 Korea - Daegu Tel: 82-53-744-4301 Korea - Seoul Tel: 82-2-554-7200 Malaysia - Kuala Lumpur Tel: 60-3-7651-7906 Malaysia - Penang Tel: 60-4-227-8870 Philippines - Manila Tel: 63-2-634-9065 Singapore Tel: 65-6334-8870 Taiwan - Hsin Chu Tel: 886-3-577-8366 Taiwan - Kaohsiung Tel: 886-7-213-7830 Taiwan - Taipei Tel: 886-2-2508-8600 Thailand - Bangkok Tel: 66-2-694-1351 Vietnam - Ho Chi Minh Tel: 84-28-5448-2100	Austria - Wels Tel: 43-7242-2244-39 Fax: 43-7242-2244-393 Denmark - Copenhagen Tel: 45-4485-5910 Fax: 45-4485-2829 Finland - Espoo Tel: 358-9-4520-820 France - Paris Tel: 33-1-69-53-63-20 Fax: 33-1-69-30-90-79 Germany - Garching Tel: 49-8931-9700 Germany - Haan Tel: 49-2129-3766400 Germany - Heilbronn Tel: 49-7131-72400 Germany - Karlsruhe Tel: 49-721-625370 Germany - Munich Tel: 49-89-627-144-0 Fax: 49-89-627-144-44 Germany - Rosenheim Tel: 49-8031-354-560 Israel - Ra'anana Tel: 972-9-744-7705 Italy - Milan Tel: 39-0331-742611 Fax: 39-0331-466781 Italy - Padova Tel: 39-049-7625286 Netherlands - Drunen Tel: 31-416-690399 Fax: 31-416-690340 Norway - Trondheim Tel: 47-72884388 Poland - Warsaw Tel: 48-22-3325737 Romania - Bucharest Tel: 40-21-407-87-50 Spain - Madrid Tel: 34-91-708-08-90 Fax: 34-91-708-08-91 Sweden - Gothenberg Tel: 46-31-704-60-40 Sweden - Stockholm Tel: 46-8-5090-4654 UK - Wokingham Tel: 44-118-921-5800 Fax: 44-118-921-5820